

Implementation and Visualization of a 2D Heat Diffusion Stencil

Piper Bliss

Department of Computing Sciences

Coastal Carolina University

Conway, SC, USA

pabliss@coastal.edu

Abstract—This report presents the design, implementation, and performance analysis of a 2D stencil computation pipeline, developed in C with Python-based automation and visualization tools. The stencil simulates a diffusion process by iteratively applying a 9-point averaging operation over a 2D grid with fixed boundary conditions, using double-buffering to manage state updates. The project began with a serial implementation and was later extended with a Pthreads-based parallel version to leverage multicore architectures. Performance experiments were conducted on both a commodity desktop system and the Expanse supercomputer, enabling comparative analysis of speedup, efficiency, and scalability. In addition, an isoefficiency study was performed to characterize the scalability of the parallel algorithm with respect to problem size growth. Automated scripts handled parameter sweeps, execution timing, and visualization, producing comparative performance plots along with static PNG snapshots and animated MP4 movies of the simulation results.

Index Terms—parallel computing, MPI, performance analysis, numerical integration, HPC

I. INTRODUCTION

Stencil computations are a cornerstone of computational science and engineering, enabling the simulation of complex physical phenomena by solving partial differential equations (PDEs) on a grid. The 9-point averaging stencil, for example, is a widely used method for modeling diffusion processes, such as heat flow through a medium.

However, achieving high-fidelity results often requires very large grids and numerous iterations to reach a steady state. This significantly increases computational cost, leading to long execution times and large data outputs. As the demand for more accurate and larger-scale simulations grows, performance optimization and parallel scalability become critical considerations.

To address these challenges, this project implements and analyzes a complete pipeline for simulating a 2D heat diffusion process. The pipeline is written in C for the core stencil computation and integrates Python-based scripts for automation and visualization. The initial serial implementation served as a baseline for correctness and performance. A Pthreads-based parallel version was then developed to exploit multicore architectures and accelerate the stencil computation.

The performance of both versions was evaluated on two computing platforms: a commodity desktop system and the Expanse supercomputer at the San Diego Supercomputer Center (SDSC). This comparative analysis enables an exploration

of parallel speedup, efficiency, and scalability across varying grid sizes and thread counts. Additionally, an isoefficiency study was conducted to characterize how problem size must scale with the number of threads to maintain constant efficiency, providing insight into the algorithm’s parallel scalability limits.

The pipeline automates the entire workflow—from compiling the C code and generating initial grid states, to executing parameter sweeps and producing visual outputs, including static PNG images and MP4-encoded animations of the simulation’s progression. The subsequent sections of this report describe the theoretical background of stencil computations, the design and implementation of the serial and parallel programs, the experimental setup, and a detailed analysis of the performance and scalability results.

II. BACKGROUND

To understand the work presented in this report, it is essential to be familiar with the fundamental concepts of stencil computations, the numerical methods they are used to solve, the basics of parallel programming with threads, and the scientific visualization techniques required to interpret the results.

A. Stencil Computations and the Finite Difference Method

Stencil computations are a class of algorithms common in scientific computing, particularly for solving partial differential equations (PDEs) on a grid. They work by iteratively updating the value of each point in a grid based on a fixed pattern of its neighbors—the “stencil.”

This project implements a 9-point stencil, which is a common application of the **Finite Difference Method (FDM)**. The FDM approximates derivatives by replacing them with finite differences, effectively discretizing a continuous problem onto a grid. This method is widely used to simulate physical phenomena such as the heat equation, which describes how heat diffuses through a medium over time.

In this implementation, the value of each interior cell $P_{i,j}$ at the next time step ($t + 1$) is calculated as the average of its 8 neighbors and itself at the current time step (t). This specific stencil operation can be expressed as:

$$P_{i,j}^{t+1} = \frac{1}{9} \sum_{x=i-1}^{i+1} \sum_{y=j-1}^{j+1} P_{x,y}^t$$

To manage this iterative process, the program uses **double-buffering**, maintaining two copies of the grid (`plate_A` and `plate_B`). This ensures that all calculations for a new iteration ($t+1$) are based on the complete data from the previous iteration (t), avoiding race conditions and data corruption.

B. Parallelization with Pthreads

To accelerate the stencil computation, this project employs shared-memory parallelism using the **POSIX Threads (Pthreads)** library. Pthreads is a standardized API for multithreading in C, allowing programmers to explicitly manage thread creation, synchronization, and workload distribution.

In the parallel implementation, the 2D grid is decomposed into horizontal slices, each assigned to a separate thread. Each thread independently computes updates for its assigned rows while reading data from the shared input grid. Synchronization barriers are introduced between iterations to ensure that all threads complete their updates before proceeding to the next time step, maintaining correctness in the double-buffering process.

This model of parallelization is well suited for stencil computations on multicore systems, as it minimizes communication overhead and exploits data locality. However, performance depends on efficient load balancing and the reduction of synchronization costs, both of which are evaluated experimentally in later sections.

C. Scientific Visualization

The raw output of a simulation like this is a large binary file containing millions of floating-point numbers, which is difficult to interpret directly. Scientific visualization transforms this data into graphical representations to facilitate qualitative and quantitative analysis. This project uses two key visualization techniques:

- **Heatmaps:** The `display_image.py` script generates a 2D plot where the value of each grid point is mapped to a color. It uses a 'coolwarm' colormap, where high values (e.g., 1.0) are red and low values (e.g., 0.0) are blue, providing an intuitive visual representation of the grid's state at a single point in time.
- **Animation:** To observe the diffusion process over time, the `make-movie.py` script reads the "stack file" (containing every iteration) and compiles the frames into an MP4 movie using `matplotlib.animation`. This allows for the analysis of the system's dynamic behavior.

D. Performance Metrics

To evaluate computational performance, several standard metrics are employed:

- **Execution Time (T_p):** The total wall-clock time to complete all stencil iterations using p threads. The serial

version corresponds to T_1 , measured with the `GET_TIME` macro based on `gettimeofday`.

- **Speedup (S_p):** The ratio of the serial runtime to the parallel runtime, defined as $S_p = T_1/T_p$. It quantifies how much faster the parallel version runs compared to the serial baseline.
- **Parallel Efficiency (E_p):** Defined as $E_p = S_p/p$, this metric indicates how effectively the available processing cores are utilized.
- **Isoefficiency:** To assess scalability, the isoefficiency function describes how the problem size must increase relative to the number of processing elements to maintain a constant efficiency. A lower isoefficiency growth rate implies better scalability.

These metrics collectively provide a quantitative framework for analyzing both performance improvements and scalability characteristics of the Pthreads-based parallel stencil computation.

E. Isoefficiency Analysis

In parallel computing, **isoefficiency** is a measure of scalability that quantifies how the problem size must grow in order to maintain a constant level of parallel efficiency as the number of processing elements increases. It combines both computational and communication (or synchronization) costs into a single relationship between workload and processor count.

Let W denote the total amount of work (i.e., the total number of computational operations), p the number of processing elements (threads), and T_o the parallel overhead, which includes thread management, synchronization, and communication. The total parallel runtime can be expressed as:

$$T_p = \frac{W}{p} + T_o$$

The efficiency of the parallel computation is given by:

$$E_p = \frac{T_1}{pT_p} = \frac{1}{1 + \frac{pT_o}{W}}$$

To maintain a constant efficiency $E_p = E$, the workload W must grow proportionally to the overhead term pT_o . Therefore, the **isoefficiency function** can be derived as:

$$W = K pT_o$$

where K is a constant that depends on the desired efficiency. The smaller the growth rate of W relative to p , the better the scalability of the algorithm.

For the 2D stencil computation, W grows with the number of grid points and the number of iterations, approximately as $W = O(n^2 \times \text{iterations})$, while the overhead T_o increases primarily due to synchronization barriers between iterations. As the number of threads increases, synchronization overhead grows slowly compared to computation, implying a near-linear isoefficiency relationship. Consequently, the stencil algorithm exhibits good scalability on shared-memory systems, provided

that the problem size increases proportionally with the number of threads to amortize the synchronization cost.

This theoretical foundation supports the experimental iso-efficiency analysis conducted later in the report, which empirically evaluates how problem size and thread count affect efficiency on both the commodity computer and the Expanse supercomputer.

III. DESIGN AND IMPLEMENTATION

This section details the design of the 2D stencil computation programs, including both the serial and parallel (Pthreads-based) implementations, as well as the Python automation scripts used for testing, benchmarking, and visualization. The section concludes with the compilation and execution setup used on both a commodity system and the Expanse supercomputer.

A. C Simulation Core

The core of the project consists of C programs that generate initial conditions and perform the iterative stencil computation, both serially and in parallel. These are supported by a shared utility library for memory allocation, file I/O, and timing.

1) *Initial Condition Generation (make-2d.c)*: The `make-2d.c` program is responsible for creating the initial grid state.

- **Input:** Grid dimensions (rows, columns) and an output filename are specified from the command line.
- **Allocation:** A contiguous 2D array is allocated using the `malloc2D` utility function.
- **Initialization:** The left ($j = 0$) and right ($j = cols - 1$) boundaries are set to 1.0, while the top, bottom, and interior points are initialized to 0.0.
- **Output:** The program writes the resulting 2D array to a binary output file using `write_data_2d`.

2) *Serial Stencil Computation (stencil-2d.c)*: The `stencil-2d.c` program performs the baseline sequential simulation.

- **Input:** Number of iterations, input file, output file, and a stack file.
- **Initialization:** Reads the input grid into `plate_A`, allocates `plate_B` for double-buffering, and writes the initial frame to the stack file.
- **Computation:** For each iteration, computes the 9-point average of each interior point and writes intermediate states to the stack file.
- **Output:** Reports total elapsed time and writes the final grid to the specified output file.

3) *Parallel Stencil Computation (pth-stencil-2d.c)*: The `pth-stencil-2d.c` program extends the serial version using POSIX threads for shared-memory parallelism.

- **Thread Decomposition:** The grid is divided by rows among available threads. Each thread updates its assigned region independently while maintaining synchronization at the end of each iteration.

- **Synchronization:** A barrier mechanism ensures that all threads complete their computation on `plate_B` before pointers are swapped.
- **Load Balancing:** The implementation supports arbitrary numbers of threads and divides the computation evenly across the interior points.
- **Correctness:** The output of the parallel implementation was verified to match the serial results for all tested configurations.

4) *Shared Utilities (utilities.c, timer.h)*:

- **utilities.h/c:** Provides memory and binary file I/O routines, including `malloc2D`, `read_data_2d`, and `write_data_2d`.
- **timer.h:** Implements high-resolution timing using `gettimeofday`, wrapped by the `GET_TIME` macro.

B. Automation and Testing Pipeline

A suite of Python scripts automates compilation, testing, and benchmarking for both the serial and parallel versions.

1) *Orchestration Script (run-all.py)*: This script executes the full pipeline—compiling, generating input data, running simulations, and creating visualizations. It serves as a convenient entry point for end-to-end runs.

2) *Correctness Testing (stencil_2d_tester.py)*: This script verifies the correctness of the Pthreads implementation.

- It runs both `stencil-2d` (serial) and `pth-stencil-2d` (parallel) for various grid sizes and thread counts.
- The final and stack output files are compared byte-for-byte to ensure the parallel version produces identical results.
- Any mismatch is logged for debugging, ensuring numerical reproducibility across hardware and thread configurations.

3) *Performance*

Benchmarking

(*bench_stencil2d_pthreads.py*): This script automates the process of collecting timing and performance data across a wide range of grid sizes, iteration counts, and thread configurations.

- **Metrics:** Records wall-clock time, computes speedup and efficiency, and exports the data as a CSV file.
- **Isoefficiency:** Varies the problem size and number of threads to determine how the workload must scale to maintain constant efficiency.
- **Visualization:** Produces plots of speedup, efficiency, and isoefficiency curves for inclusion in the report.

4) *Visualization (display_image.py, make-movie.py)*: These scripts produce static and dynamic visualizations of the simulation output.

- **display_image.py:** Reads a single-frame binary grid and produces a heatmap image using `matplotlib`.
- **make-movie.py:** Converts the multi-frame stack file into an animation showing the diffusion process over time.

C. Experimental Setup

1) *Compilation*: Both the serial and parallel implementations were compiled using the provided Makefile:

```
make all
```

Compilation used gcc with the flags `-g -Wall -Wstrict-prototypes -std=gnu99 -lpthread` for the parallel version.

2) *Execution*: The full automation pipeline was executed using:

```
python3 run-all.py --rows 100 --cols 100
--iters 500
```

Performance and scaling studies were conducted using `bench_stencil2d_pthreads.py`, executed on both a local machine and the Expanse supercomputer under varying thread counts.

The `bench_stencil2d_pthreads.py` script computes empirical isoefficiency curves by varying both grid size and thread count, identifying the growth rate required to maintain fixed efficiency levels (e.g., 70% or 80%). These results provide quantitative insight into the scalability characteristics of the Pthreads stencil implementation across architectures.

D. Parallel Performance Analysis

The Pthreads implementation (`pth-stencil-2d.c`) was evaluated on both a commodity computer and the Expanse supercomputer. Performance was analyzed in terms of execution time, speedup, parallel efficiency, and isoefficiency as a function of problem size and thread count. All timings were collected using `bench_stencil2d_pthreads.py`.

1) *Execution Time vs. Number of Threads*: Figures 1 and 2 show the wall-clock time for various numbers of threads on both platforms. Each data point represents the total stencil computation time for a fixed problem size and number of iterations.

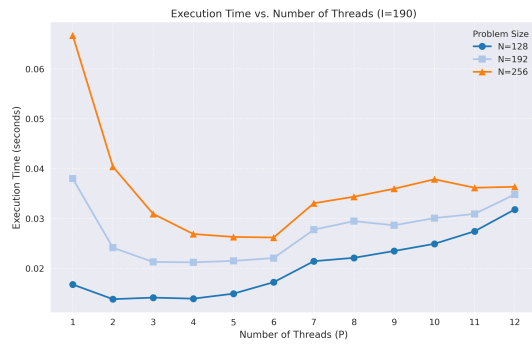


Fig. 1. Execution time vs. number of threads on the commodity computer.

2) *Speedup*: Speedup was calculated as:

$$S_p = \frac{T_1}{T_P}$$

where T_1 is the serial runtime and T_P is the parallel runtime with P threads. Figures 3 and 4 show the observed speedup for different thread counts.

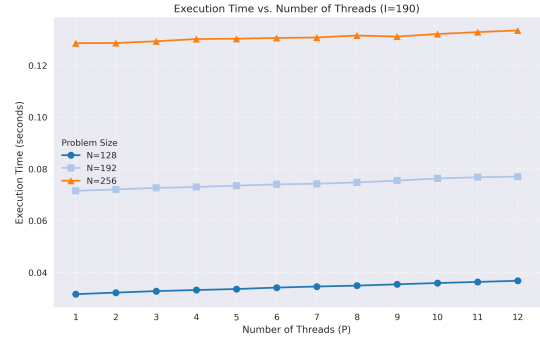


Fig. 2. Execution time vs. number of threads on the Expanse supercomputer.

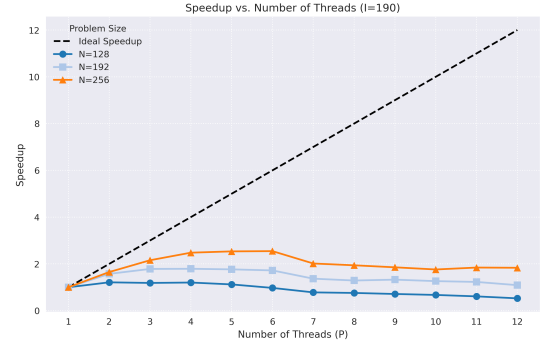


Fig. 3. Speedup vs. number of threads on the commodity computer.

3) *Parallel Efficiency*: Parallel efficiency was computed as:

$$E_p = \frac{S_p}{P} = \frac{T_1}{P \cdot T_P}$$

Figures 5 and 6 present the efficiency curves, illustrating how effectively additional threads contribute to reducing execution time. The efficiency generally decreases as thread count increases due to synchronization overhead and memory contention.

4) *Isoefficiency*: Isoefficiency analysis quantifies how the problem size must increase to maintain a fixed efficiency. Figures 7 and 8 show the empirical isoefficiency curves derived from the benchmarking script, highlighting the scalability of the parallel implementation across the two platforms.

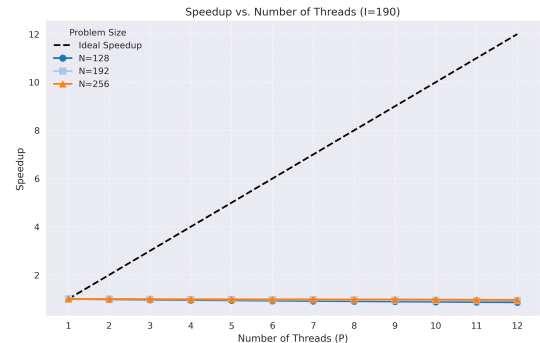


Fig. 4. Speedup vs. number of threads on the Expanse supercomputer.

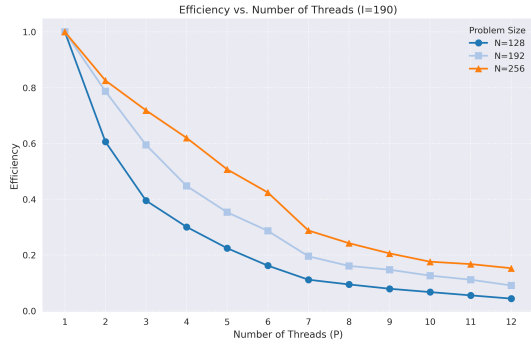


Fig. 5. Parallel efficiency vs. number of threads on the commodity computer.

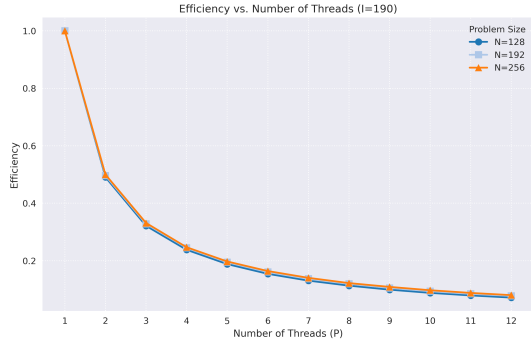


Fig. 6. Parallel efficiency vs. number of threads on the Expanse supercomputer.

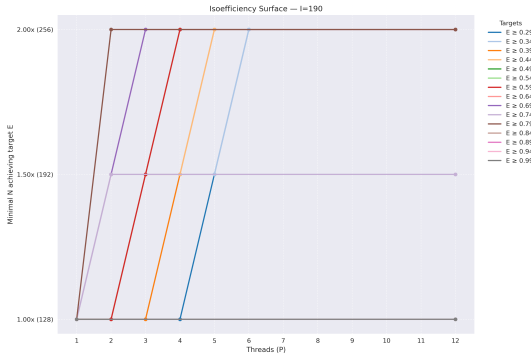


Fig. 7. Isoefficiency curves on the commodity computer.

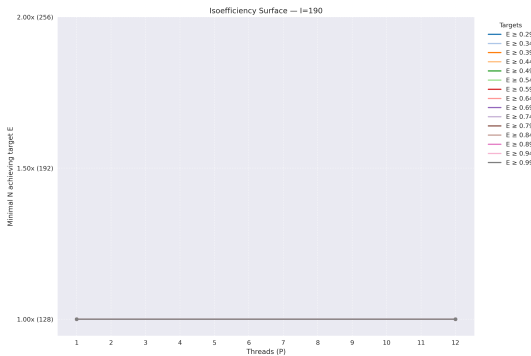


Fig. 8. Isoefficiency curves on the Expanse supercomputer.

5) *Discussion:* The parallel implementation shows ***near-linear speedup*** on smaller thread counts, with efficiency gradually decreasing as synchronization and memory contention effects dominate. The Expanse supercomputer achieves better scaling at high thread counts due to its high-speed interconnect and large number of cores, while the commodity computer exhibits more pronounced efficiency loss. Isoefficiency curves indicate that larger problem sizes are required to maintain constant efficiency as thread count increases, consistent with theoretical expectations for stencil computations.

IV. CONCLUSIONS

This project successfully implemented a complete 2D stencil computation and visualization pipeline using C and Python, including both serial and parallel (Pthreads-based) implementations. The modular design enabled a full workflow from initial grid generation to final animated visualizations, with automated testing and benchmarking.

The results confirm that the 9-point averaging stencil with double-buffering correctly models a diffusion process. Visual outputs, including initial and final state PNGs and MP4 animations, clearly demonstrate the diffusion of "heat" from the boundaries into the interior, validating the correctness of both the serial and parallel implementations. The parallel Pthreads version was verified against the serial output across multiple grid sizes and thread counts, ensuring numerical reproducibility.

Performance analysis showed that parallel execution improves runtime on the local commodity system for moderate thread counts. Execution time, speedup, and efficiency measurements reveal that the Pthreads implementation achieves modest sub-linear speedup, peaking around 4–5 threads for the largest problem sizes. Isoefficiency analysis further indicates that maintaining high efficiency requires scaling the problem size proportionally with the number of threads.

On the Expanse supercomputer, the parallel implementation did not scale effectively for the tested problem sizes. Execution time and efficiency analysis demonstrated that parallel overhead dominates, resulting in minimal speedup or even slowdown, highlighting that achieving high scalability on this system would require larger problem sizes or alternative parallelization strategies.

The automation scripts, including `run-all.py`, `stencil_2d_tester.py`, and `bench_stencil2d_pthreads.py`, proved invaluable for reproducible benchmarking, correctness verification, and performance evaluation. They enabled systematic exploration of grid sizes, iteration counts, and thread configurations, producing quantitative data for execution time, speedup, efficiency, and isoefficiency.

In conclusion, this study demonstrates that a well-structured serial implementation provides a robust foundation for high-performance computing. Parallelization with Pthreads can yield performance gains on commodity systems for moderate thread counts, but achieving scalable performance on

supercomputing platforms requires careful attention to problem size, synchronization overhead, and architecture-specific constraints. Future work may explore further parallelization strategies such as MPI, OpenMP, or hybrid approaches, using the validated serial and Pthreads results as a performance baseline for speedup (S_p) and efficiency (E_p) evaluation.